opacity:1;*top:-2px;*left:-5px
top:-4px\0/;left:-6px\0/;
inline-box;display:inline-block;
display:block;list-style:none;
inline-block;line-height:27px;
pointer;display:block;

Strings

Plan für die Woche

Montag	Dienstag	Mittwoch	Donnerstag	Freitag
• StringPool	• Collections	• Prüfung	• Collections	• Prüfung

Lernziele

- Verstehen, was Strings sind
- Deklarieren und Initialisieren von Strings
- Was heißt Immutable?
- Nutzen der Methoden der String-Klasse
- Escape-Sequenzen

Strings

Was sind Strings?

- Strings ist eine Folge von Zeichen (chars), die als Text interpretiert wird
- ein String kann Buchstaben, Zahlen, Leerzeichen und Sonderzeichen enthalten
- sie sind Objekte der Klasse java.lang.String
- sobald ein String erstellt wurde, ist er unveränderbar (immutable)

„Hallo“
„Java“
„12345“
„Hallo mein Name ist“

Erzeugen eines Strings

– Initialisieren eines Strings:

```
String s = "Hallo";
```

```
String s1 = new String("Hallo");
```

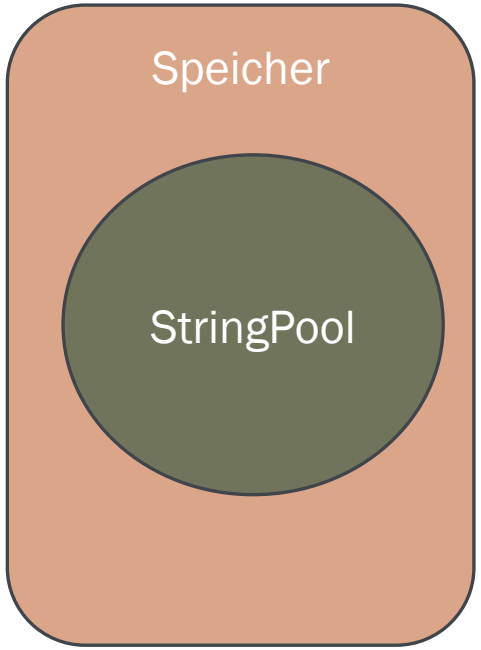
Sehr selten! Da ineffizient

Je nach Initialisierung passieren im Hintergrund verschiedene Sachen…

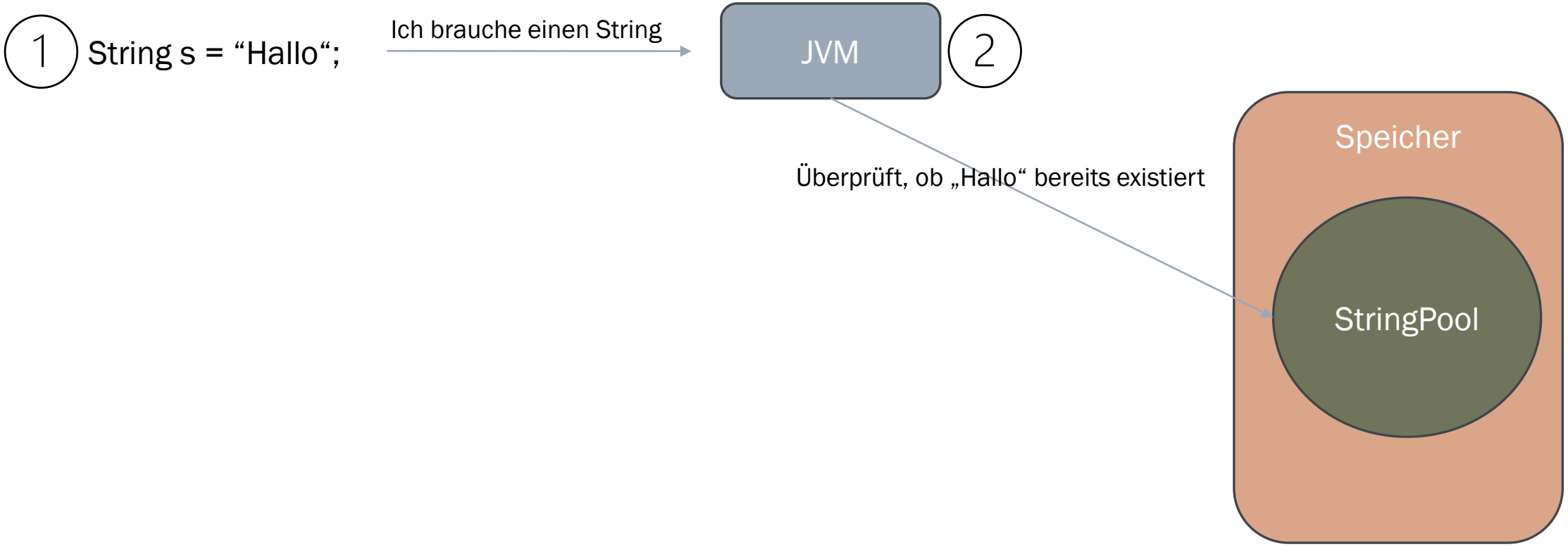
StringPool

StringPool

- ist ein spezieller Bereich im Speicher, in dem einzigartige String-Literale gespeichert werden
- dadurch soll Speicher gespart und der Code effizienter werden

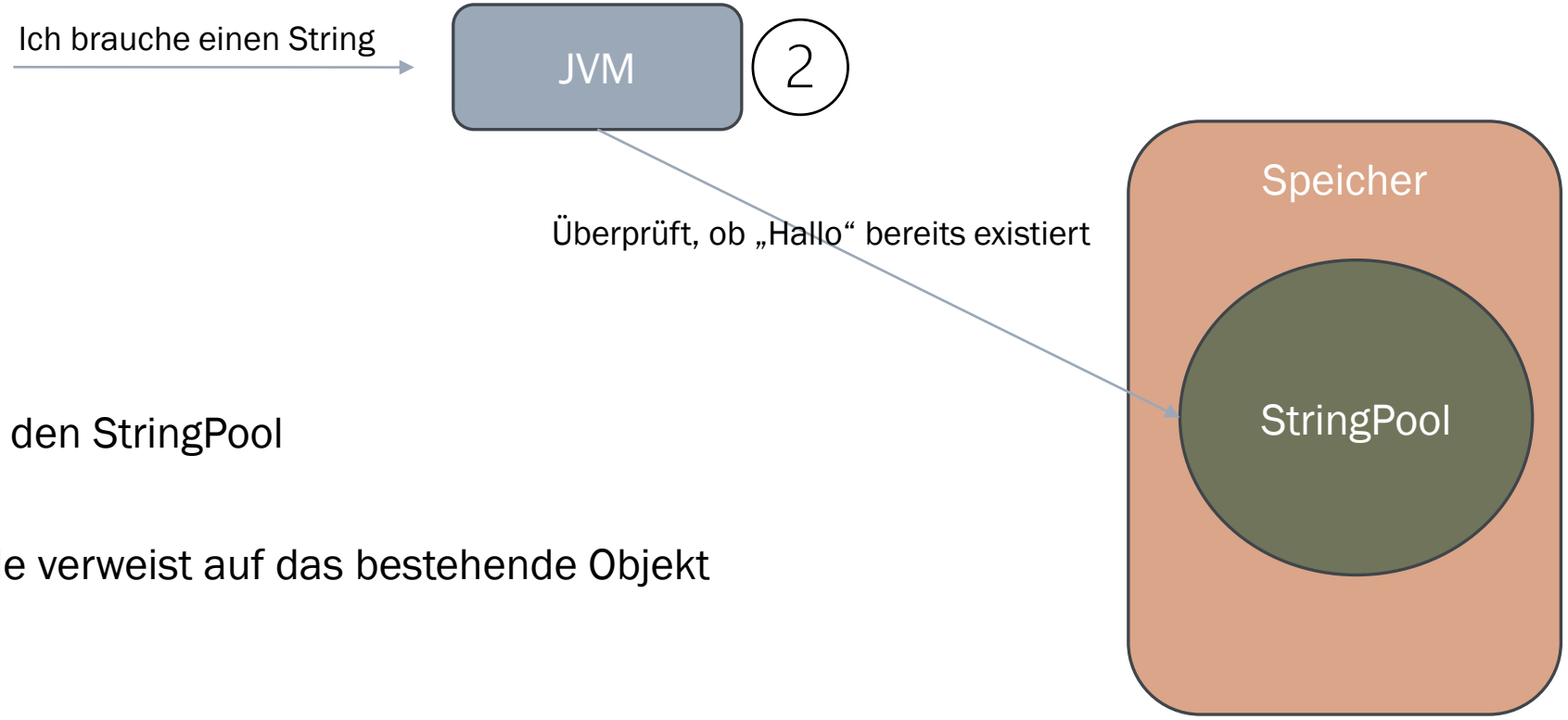


StringPool


1
String s = “Hallo“;
Ich brauche einen String
JVM
2
Überprüft, ob „Hallo“ bereits existiert
Speicher
StringPool

StringPool

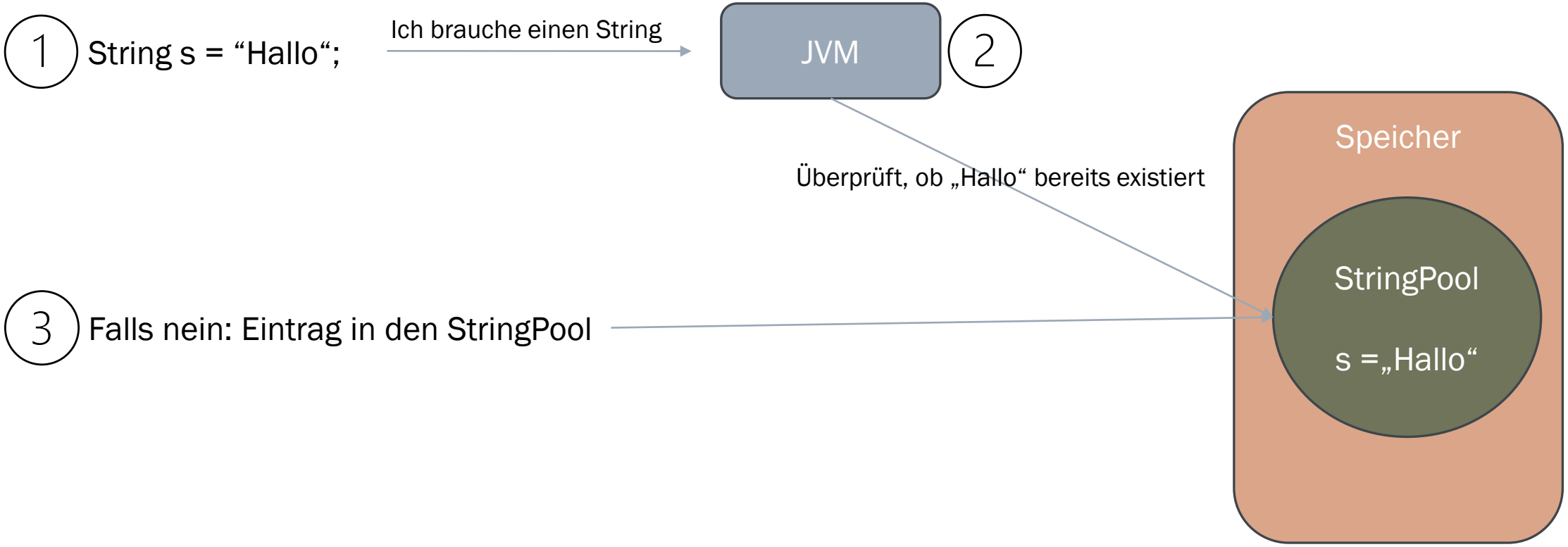
1 String s = “Hallo“;



3 Falls nein: Eintrag in den StringPool

3 Falls ja: neue Variable verweist auf das bestehende Objekt

StringPool


1
String s = “Hallo“;
Ich brauche einen String
JVM
2
Überprüft, ob „Hallo“ bereits existiert
Speicher
StringPool
s =„Hallo“
3
Falls nein: Eintrag in den StringPool

StringPool

1 String s = “Hallo“;

Ich brauche einen String

JVM

2

Überprüft, ob „Hallo“ bereits existiert

Speicher

StringPool

s =„Hallo“

3 Falls nein: Eintrag in den StringPool

3 Falls ja: neue Variable verweist auf das bestehende Objekt

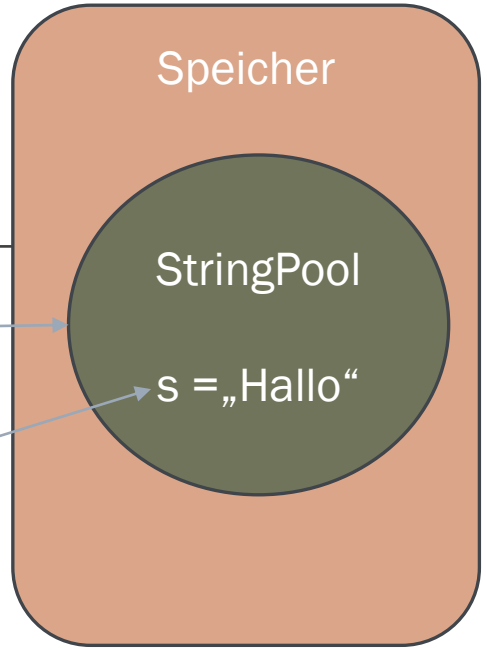
String s1 = “Hallo“

StringPool

3 Falls nein: Eintrag in den StringPool

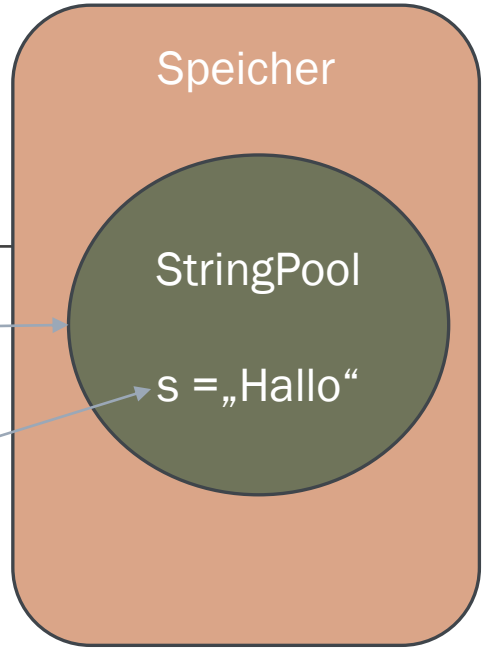
3 Falls ja: neue Variable verweist auf das bestehende Objekt

String s1 = “Hallo“



Was ergibt s == s1 ?

StringPool



3 Falls nein: Eintrag in den StringPool

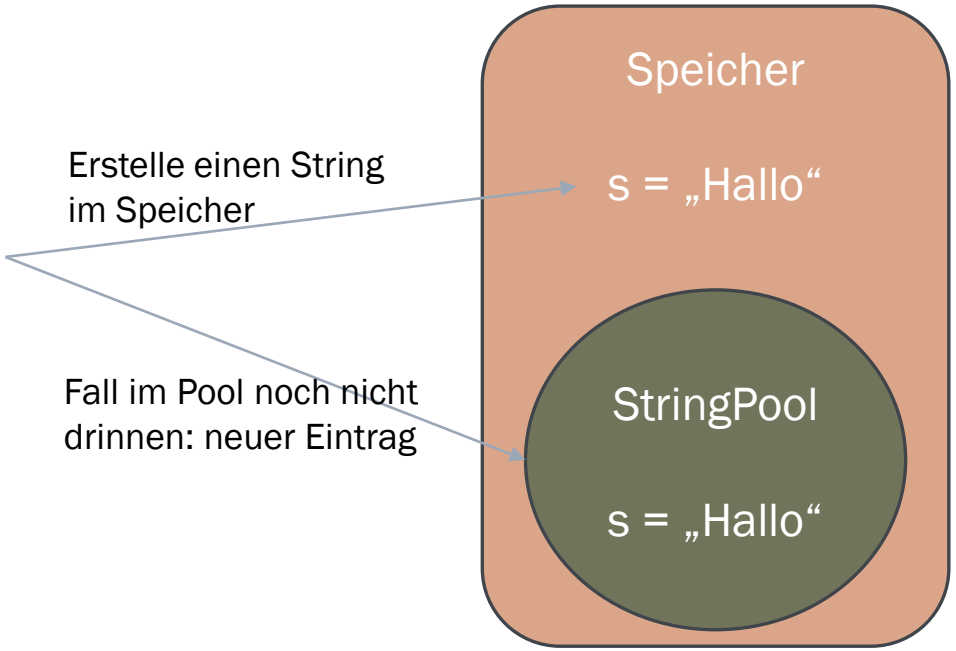
3 Falls ja: neue Variable verweist auf das bestehende Objekt

String s1 = “Hallo“

Was ergibt s.equals(s1) ?

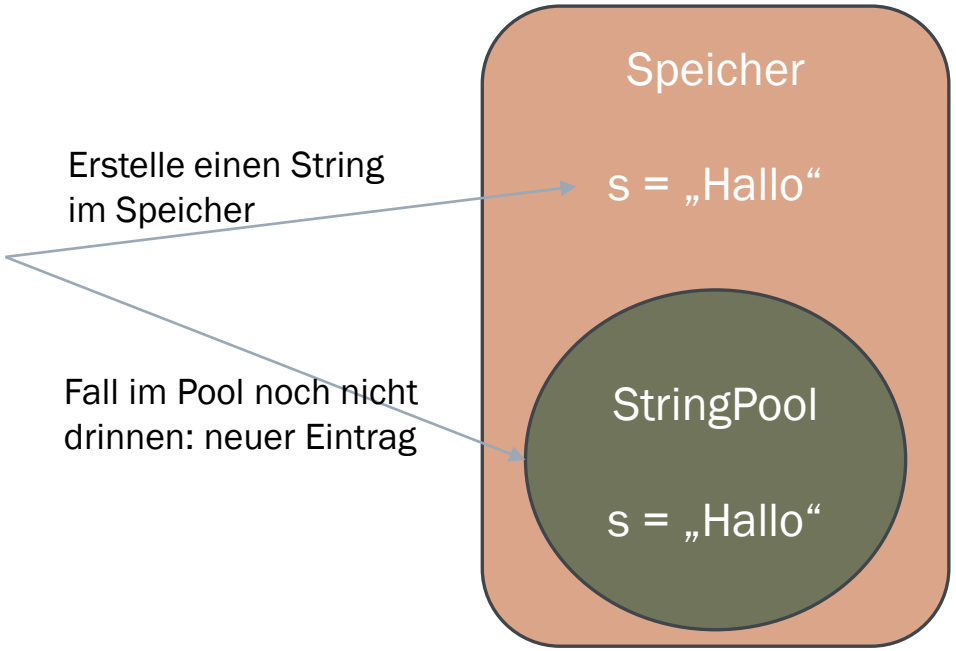
StringPool

1 String s = new String(“Hallo“);



StringPool

1 String s = new String(“Hallo“);

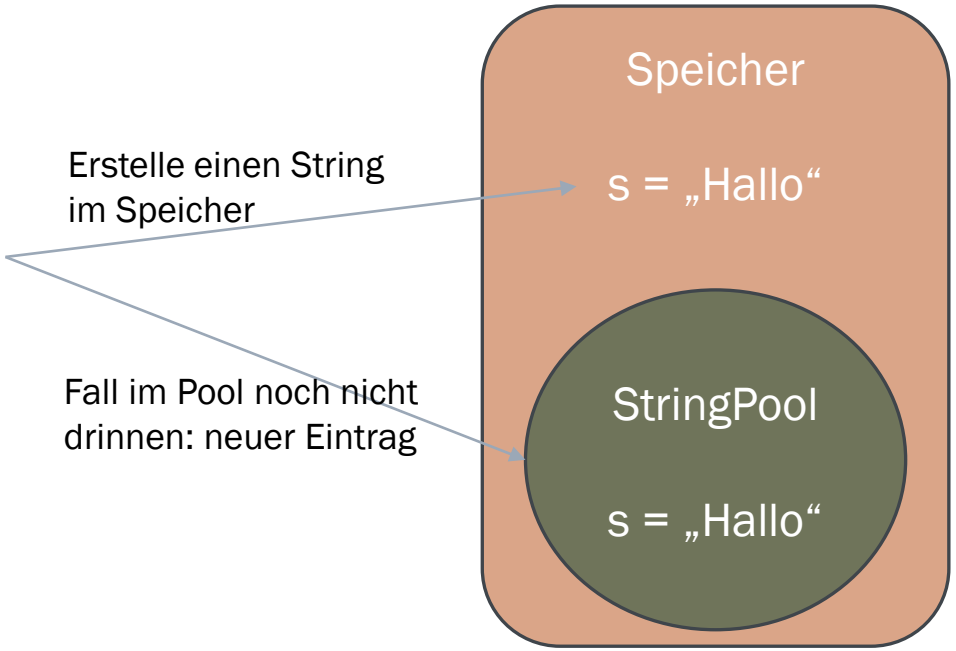


Wie viele Objekte werden erzeugt?

StringPool

1 String s = new String(“Hallo“);

String s1 = new String(“Hallo“);



Was passiert, wenn man nun s == s1 macht?

Strings verwenden

Concatenation (+)

- Strings kann man mittels „+“ verbinden
  - Die anderen Operatoren wie *,/,… funktionieren nicht
- das Ergebnis landet **nicht** im StringPool, sondern im Speicher (Heap)

```
String str1 = "Hallo";
String str2 = "Welt";
String str3 = str1 + " " + str2;
```

Landet auf dem Heap

Concatenation (+)

- bei Concatenation muss man auf folgendes achten:
  - **Numerische Operanden werden addiert, solange vor ihnen kein String steht**

Was ist die Ausgabe?

- bei Concatenation muss man auf folgendes achten:
  - Numerische Operanden werden addiert, solange vor ihnen kein String steht

```
System.out.println(1 + 2);
System.out.println(1 + 2 + " Hallo");
System.out.println("Hallo" + " Tschüss");
System.out.println("Hallo" + 1 + 2);
```

A. 3
3 Hallo
Hallo Tschüss
Hallo 3

B. 3
12 Hallo
Hallo Tschüss
Hallo 3

C. 3
3 Hallo
Hallo Tschüss
Hallo12

D. 3
12Hallo
Hallo Tschüss
Halo12

Was ist die Ausgabe?

- bei Concatenation muss man auf folgendes achten:
  - Numerische Operanden werden addiert, solange vor ihnen kein String steht

```
String s = "Ciao";
int i = 10;
System.out.println(1 + 2 + i + s);
```

A. 12 10Ciao

B. 310Ciao

C. 13Ciao

D. 1210Ciao

Methoden

- length():
  - Gibt die Länge der Characters zurück, die im String enthalten sind

```
String str1 = "Hallo";
System.out.println(str1.length());
```

str1.length() ergibt 5

Methoden

- charAt(int index)
  - gibt den Character am spezifischen index aus
  - Index startet bei 0!

```
String str1 = "Hallo";
System.out.println(str1.charAt(4));
```

Ergibt o

```
String str1 = "Hallo";
System.out.println(str1.charAt(5));
```

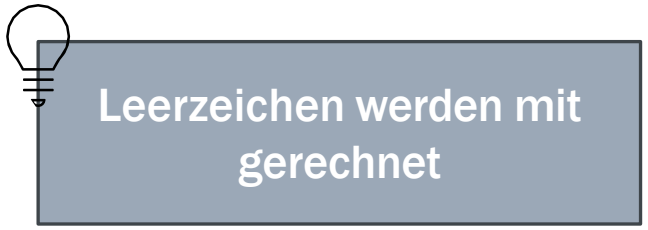
Exception! StringIndexOutOfBoundsException

Methoden

- indexOf(String str)
  indexOf(int ch)

```
String str1 = "Hallo";
System.out.println(str1.indexOf("l"));
```

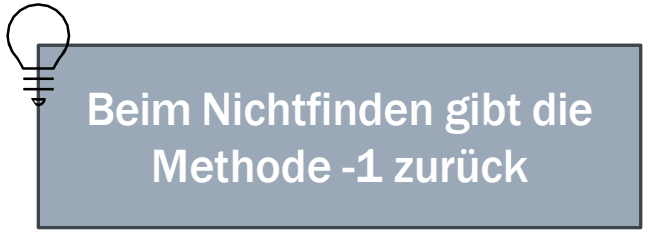
Ergibt 2



- indexOf(String str, int fromIndex)
  indexOf(int ch, int fromIndex)

```
String str1 = "Hallo ma Frenda";
System.out.println(str1.indexOf(„a", 8));
```

Ergibt 14



Methoden

- substring(int beginIndex)
  - Sucht nach Characters in einem String
  - Gibt einen Teil des Strings zurück
  - Index fängt bei 0 an!

```
String str1 = "Hallo";
System.out.println(str1.substring(3));
```

Ergibt lo

- substring(int beginIndex, int endIndex)
  - endIndex als exklusiv betrachten!

```
String str1 = "Hallo ma Frenda";
System.out.println(str1.substring(2, 10));
```

Ergibt llo ma F

Methoden

- toLowerCase()/toUpperCase():
  - toLowerCase() : alle Characters des Strings werden klein geschrieben
  - toUpperCase() : alle Characters des Strings werden groß geschrieben

```
String str1 = "Hello ma Frenda";
System.out.println(str1.toLowerCase());        //hello ma frenda
System.out.println(str1.toUpperCase());        //HELLO MA FRENDA
```

Methoden

- equals():
  - Gibt einen boolean zurück
  - Vergleicht die Inhalte von Strings
  - Case-sensitive: achtet auf die Groß-/Kleinschreibung des Strings

```
System.out.println("abc".equals("abc"));                    //true
System.out.println("abc".equals("ABC"));                    //false
System.out.println("abc".equalsIgnoreCase("ABC"));          //true
```

- equalsIgnoreCase():
  - Gibt einen boolean zurück
  - Vergleicht ebenfalls die Inhalte von Strings
  - Nicht case-sensitive: achtet NICHT auf die Groß-/Kleinschreibung des Strings

Methoden

- startsWith(String prefix):
  - Gibt einen boolean zurück, wenn ein String mit dem übergebenen String anfängt
  - Case-sensitive

```
System.out.println("abc".startsWith("a"));        //true
System.out.println("abc".startsWith("A"));        //false
System.out.println("abc".endsWith("a"));          //false
System.out.println("abc".endsWith("c"));          //true
```

- endsWith(String suffix)
  - Gibt einen boolean zurück, wenn ein String mit dem übergebenen String endet
  - Case-sensitive

Methoden

- contains(String str):
  - Überprüft, ob der übergebene String im ursprünglichen String enthalten
  - Case-sensitive

```
System.out.println("abc".contains("a"));        //true
System.out.println("abc".contains("A"));        //false
```

Methoden

- replace(char oldChar, char newChar)
  replace(CharSequence oldChar, CharSequence newChar):
  - Sucht nach dem alten Character oder alten CharacaterSequence und ersetzt ihn mit einem neuen

```
System.out.println("ababcc".replace("a", "A"));          //AbAbcc
```

Methoden

- **trim():**
  - **Löscht die Leerzeichen vor und nach einem String**

```
System.out.println(„     HALLO     mein name ist Anita      ".trim());    //HALLO   mein name ist Anita
```

compareTo

Methoden

0 -> die String sind inhaltsgleich
>0 -> der aktuelle String kommt alphabetisch nach dem anderen
<0 -> der aktuelle String kommt vor dem anderen

- **compareTo():**
  - **Vergleicht zwei Strings lexiografisch**
  - **Liefert einen int-Wert, der angibt, ob der aktuelle String vor, gleich oder nach einem anderen String liegt**

a = „A P F E L“

a.compareTo(b)

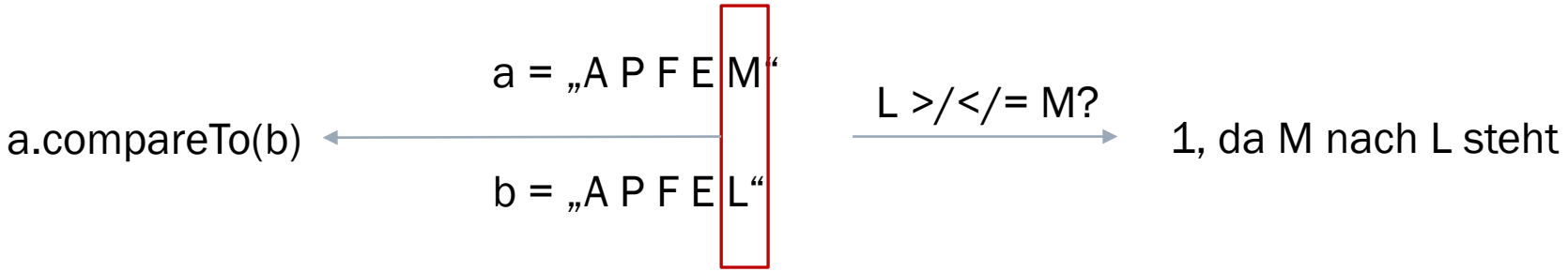
b = „B A N A N E“

A >/</= B?

-1, da A vor B steht

Methoden

- compareTo():
  - Vergleicht zwei Strings lexiografisch
  - Liefert einen int-Wert, der angibt, ob der aktuelle String vor, gleich oder nach einem anderen String liegt



Vorgehen

- Die Methode vergleicht **Zeichen für Zeichen** die beiden Strings.
- Jeder Buchstabe wird nach seinem **Unicode-Wert** bewertet.
- Sobald ein unterschiedliches Zeichen gefunden wird, wird die Differenz der Unicode-Werte zurückgegeben
- Wenn alle verglichenen Zeichen gleich sind, wird die **Länge** der Strings verglichen

Zusammenfassung

Methode	Beschreibung	Beispiel
length()	Gibt Länge zurück	"Test".length() → 4
charAt(i)	Zeichen an Position i	"Java".charAt(1) → 'a'
substring(a,b)	Teilstring	"Hallo".substring(1,4) → "all"
toUpperCase()	Großbuchstaben	"java".toUpperCase()" → "JAVA"
trim()	Entfernt Leerzeichen	" hi ".trim()" → "hi"
replace()	Ersetzt Zeichen	"Test".replace('T','B') → "Best"
split()	Zerlegt String in Array	"a,b".split(",") → ["a","b"]

Method Chaining

- die Methoden müssen nicht einzeln aufgerufen werden
- die Methoden können verkettet werden

```
public class MethodChainingExample {
    public static void main(String[] args) {
        String result = "   Hallo, Java!   "
                .trim()
                .replace("Java", "Welt")
                .toUpperCase()
                .substring(0, 10);
        System.out.println(result); // Gibt "HALLO, WEL" aus
    }
}
```

Immutable

Immutable

- sobald ein String-Objekt erzeugt wurde, kann es nicht verändert werden
  - Bei Veränderungen wird ein neues String-Objekt im Heap erzeugt

```
public class ImmutableStringExample {
    public static void main(String[] args) {
        String original = "Hallo";
        original.concat("Ola");
        System.out.println("Original: " + original);
        //Ausgabe : Hallo
    }
}
```

Immutable

- sobald ein String-Objekt erzeugt wurde, kann es nicht verändert werden
  - Bei Veränderungen wird ein neues String-Objekt im Heap erzeugt
- man muss die Veränderung einer neuen Variable zuweisen

```
public class ImmutableStringExample {
    public static void main(String[] args) {
        String original = "Hallo";
        String modified = original.concat(" Welt!");

        System.out.println("Modifiziert: " + modified);
        //Ausgabe: Hallo Welt!
    }
}
```

Aufgabe ca. 25-30 Minuten

Schreibe ein Java-Programm, das eine Benutzer-Eingabe verarbeitet und verschiedene String-Methoden darauf anwendet. Dein Programm soll:

1. Eingabe einlesen:
   - Frage den Benutzer nach einem beliebigen Satz.
   - Frage den Benutzer nach einem Wort, das im Satz vorkommen könnte.

2. Analysen und Operationen durchführen:
   - Prüfe, ob der Satz mit „Hallo“ beginnt oder mit „!“ endet.
   - Prüfe, ob der Satz das gesuchte Wort enthält (case-insensitive).
   - Ersetze alle Leerzeichen im Satz durch Unterstriche _ und gib den veränderten Satz aus.
   - Ermittle die Länge des Satzes und gib sie aus.
   - Falls das gesuchte Wort enthalten ist, gib die Position des ersten Auftretens aus. Falls nicht, gib eine entsprechende Meldung aus.
   - Schneide den Satz so zu, dass nur die ersten 10 Zeichen übrig bleiben, und gib das Ergebnis aus.
   - Entferne mögliche Leerzeichen am Anfang und Ende des Satzes und gib das bereinigte Ergebnis aus.
   - Gib das erste Zeichen des Satzes aus.

Formatierte Strings

Formatierte Strings

- wenn man Texte mit Variablen ausgeben möchte, entstehen schnell unübersichtliche Ausdrücke

```
String name = "Anna";
int alter = 25;
System.out.println(name + " ist " + alter + " Jahre alt.");
```

- Java bietet formatierte String an, mit denen man Werte strukturiert und präzise einfügen kann

String.format()

- String.format() erstellt einen neuen String mit Platzhaltern, die durch Werte ersetzt werden

```
String name = "Anna";
int alter = 25;
String satz = String.format("%s ist %d Jahre alt.", name, alter);
System.out.println(satz);
```

- %s steht für einen String
- %d steht für eine GANZE Zahle (int)

System.out.printf()

- funktioniert ähnlich, gibt das Ergebnis aber direkt auf der Konsole aus, ohne einen neuen String zu erzeugen

```
String tier = "Katze";
int beine = 4;
System.out.printf("Eine %s hat %d Beine.%n", tier, beine);
```

- %n sorgt für einen plattformunabhängigen Zeilenumbruch (statt /n, das unter Betriebssystemen unterschiedlich sein kann)

Formatierung von Zahlen und Fließkommazahlen

```
double preis = 12.3456;
System.out.printf("Preis: %.2f Euro%n", preis);
```

- f bedeutet Fließkommazahl
- .2 bedeutet zwei Nachkommastellen

Zusammenfassung

Platzhalter	Typ / Bedeutung	Beispielwert	Ausgabe
%s	String (Text)	"Java"	Java
%d	Ganzzahl (int, long, short, byte)	42	42
%f	Fließkommazahl (float, double)	3.14159	3.141590
%.2f	Fließkommazahl mit 2 Nachkommastellen	3.14159	3.14
%c	Einzelnes Zeichen (char)	'A'	A
%b	Boolean (true/false)	true	true
%n	Zeilenumbruch (plattformunabhängig)	–	(Neue Zeile)
%x	Hexadezimaldarstellung	255	ff
%e	Wissenschaftliche Schreibweise	1234.56	1.234560e+03
%t	Datums-/Zeitformatierung (z. B. %tY, %tB)	LocalDate.now()	2025, Oktober etc.